

Database Systems Laboratory Work

Week 2: Relational Model & Keys

Complete Solutions

Tool for ER diagrams: ERDPlus-compatible notation

PostgreSQL / relational design practice

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. Example superkeys (at least six):

- {EmpID}
- {SSN}
- {Email}
- {EmpID, Name}
- {SSN, Department}
- {Email, Salary}
- {EmpID, Phone, Department}

2. Candidate keys: **EmpID, SSN, and Email**. These are reasonable minimal identifiers if each is required to be unique. Phone is not treated as a candidate key because the task gives no uniqueness rule for phone numbers.

3. Primary key choice: **EmpID** because it is short, stable, easy to index, and does not expose personal information such as SSN.

4. Can two employees have the same phone number? **Yes, unless the database adds a UNIQUE rule for Phone**. The sample rows happen to show different numbers, but sample data alone does not guarantee uniqueness.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum primary key: **(StudentID, CourseCode, Section, Semester, Year)**

- StudentID identifies the student.
- CourseCode identifies the course.
- Section is needed because the same course can have multiple sections.
- Semester is needed because the same student can take the course in different semesters.
- Year is needed because the same semester name repeats every year.

3. Additional candidate keys: **None can be guaranteed from the stated business rules.**

Task 1.2: Foreign Key Design

Child column	References	Reason
Student.AdvisorID	Professor.ProfID	A student can have a professor as advisor.
Department.ChairID	Professor.ProfID	A department chair is a professor.
Course.DepartmentCode	Department.DeptCode	Each course belongs to a department.
Enrollment.StudentID	Student.StudentID	Enrollment belongs to a student.
Enrollment.CourseID	Course.CourseID	Enrollment belongs to a course.
Professor.Department*	Department.DeptCode	Valid if Professor.Department stores the department code.

**The Professor table uses the generic name Department. This FK is valid only if that field stores the department code rather than the department name.*

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

Entity classification:

- Strong entities: Patient, Doctor, Department.
- Weak entity: HospitalRoom, because RoomNo is unique only inside a department. Its full key is (DeptCode, RoomNo).
- Associative entities: Appointment and Prescription. They connect patients and doctors and also store relationship attributes.

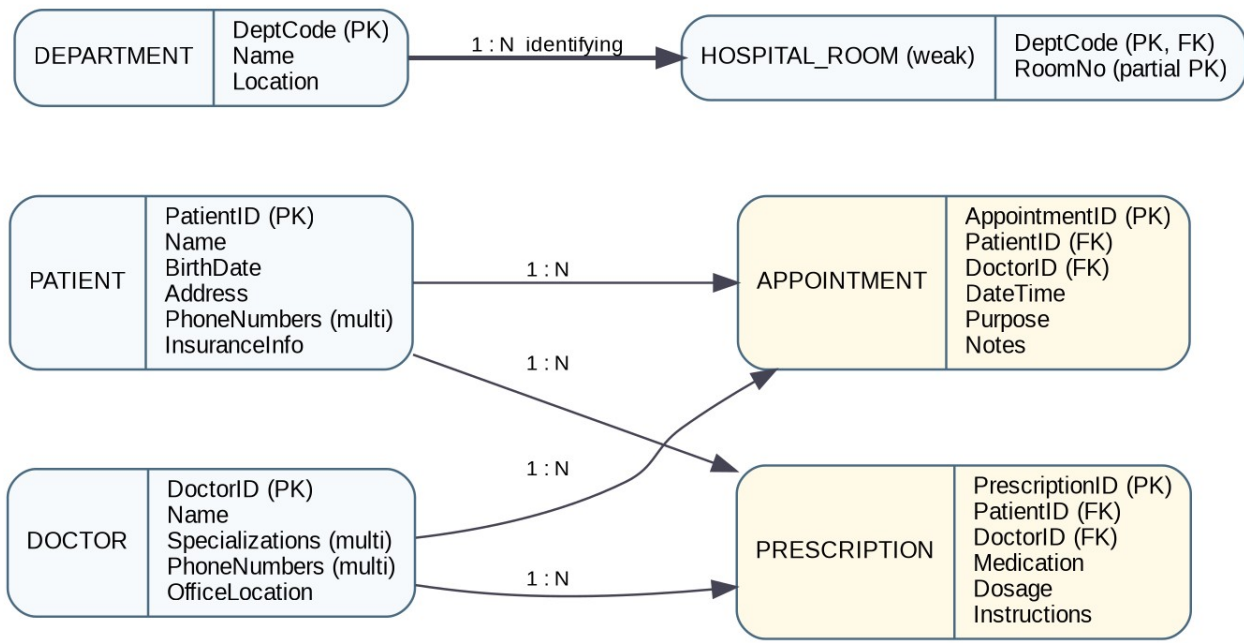
Attributes and types:

Entity	Attributes	Classification
Patient	PatientID; Name; BirthDate; Address; PhoneNumbers; InsuranceInfo	PatientID simple/key; Name composite; BirthDate simple; Address composite; PhoneNumbers multi-valued; InsuranceInfo composite. Age can be derived from BirthDate.
Doctor	DoctorID; Name; Specializations; PhoneNumbers; OfficeLocation	DoctorID simple/key; Name composite; Specializations multi-valued; PhoneNumbers multi-valued; OfficeLocation simple.
Department	DeptCode; Name; Location	All simple; DeptCode is the key.
Appointment	AppointmentID; DateTime; Purpose; Notes	Simple attributes; PatientID and DoctorID are foreign keys.
Prescription	PrescriptionID; Medication; Dosage; Instructions	Simple attributes; PatientID and DoctorID are foreign keys.
HospitalRoom	DeptCode; RoomNo	DeptCode owner key + RoomNo partial key; weak entity.

Relationships and cardinalities:

- Patient 1:N Appointment; Doctor 1:N Appointment.
- Patient 1:N Prescription; Doctor 1:N Prescription.
- Department 1:N HospitalRoom (identifying relationship).

ER Diagram - Hospital Management System



Possible relational mapping:

Patient(PatientID INT, Name VARCHAR(100), BirthDate DATE, Street VARCHAR(100), City VARCHAR(50), State VARCHAR(50), Zip VARCHAR(15), InsuranceInfo VARCHAR(150))

PatientPhone(PatientID INT, Phone VARCHAR(20))

Doctor(DoctorID INT, Name VARCHAR(100), OfficeLocation VARCHAR(100))

DoctorSpecialization(DoctorID INT, Specialization VARCHAR(100))

Department(DeptCode VARCHAR(10), DeptName VARCHAR(100), Location VARCHAR(100))

HospitalRoom(DeptCode VARCHAR(10), RoomNo VARCHAR(10))

Appointment(AppointmentID INT, PatientID INT, DoctorID INT, AppointmentTime TIMESTAMP, Purpose VARCHAR(200), Notes TEXT)

Prescription(PrescriptionID INT, PatientID INT, DoctorID INT, Medication VARCHAR(100), Dosage VARCHAR(100), Instructions TEXT)

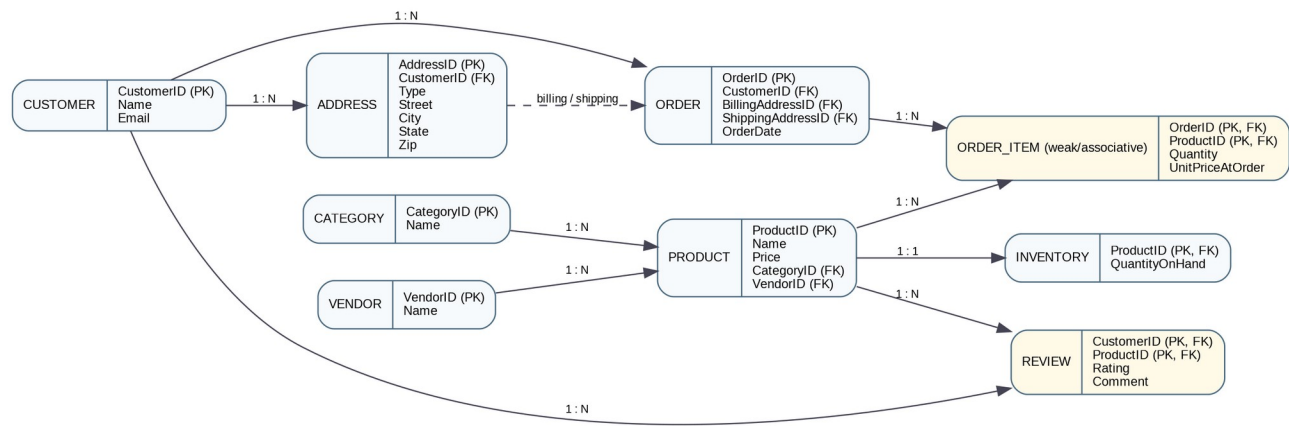
Task 2.2: E-commerce Platform

Main entities: Customer, Address, Order, Product, Category, Vendor, Inventory, Review, and OrderItem.

Weak entity: **OrderItem** is weak/associative because it exists only as part of an Order and references a Product. A practical composite key is (OrderID, ProductID), or (OrderID, LineNo) if duplicate product lines are allowed.

Many-to-many relationship with attributes: **Order M:N Product** is resolved by OrderItem, which stores Quantity and UnitPriceAtOrder. Another M:N is Customer M:N Product through Review, which stores Rating and Comment.

ER Diagram - E-commerce Platform



Relational mapping with data types:

Customer(CustomerID INT, Name VARCHAR(100), Email VARCHAR(100))

Address(AddressID INT, CustomerID INT, Type VARCHAR(20), Street VARCHAR(120), City VARCHAR(60), State VARCHAR(60), Zip VARCHAR(15))

Category(CategoryID INT, Name VARCHAR(100))

Vendor(VendorID INT, Name VARCHAR(120))

Product(ProductID INT, Name VARCHAR(150), Price DECIMAL(10,2), CategoryID INT, VendorID INT)

CustomerOrder(OrderID INT, CustomerID INT, BillingAddressID INT, ShippingAddressID INT, OrderDate TIMESTAMP)

OrderItem(OrderID INT, ProductID INT, Quantity INT, UnitPriceAtOrder DECIMAL(10,2))

Review(CustomerID INT, ProductID INT, Rating SMALLINT, Comment TEXT)

Inventory(ProductID INT, QuantityOnHand INT)

Note: The provided assignment PDF jumps from Part 2 directly to Part 4; no Part 3 is included in the source file.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional dependencies:

- StudentID -> StudentName, StudentMajor
- ProjectID -> ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate
- SupervisorID -> SupervisorName, SupervisorDept
- (StudentID, ProjectID) -> Role, HoursWorked

2. Redundancy and anomalies:

- Redundancy: StudentName and StudentMajor repeat for every project of the same student. Project and supervisor data repeat for every student assigned to the same project.
- Update anomaly: changing a supervisor department requires updating many rows.
- Insert anomaly: a new project cannot be stored until at least one student is assigned if all data must stay in this table.
- Delete anomaly: deleting the last student from a project can also remove the only stored project and supervisor information.

3. First Normal Form (1NF):

The table is in 1NF if every cell contains one atomic value and each row represents one student-project assignment. No repeating groups are shown. If a field contained multiple roles or supervisors in one cell, it would need to be split into separate rows/tables.

4. Second Normal Form (2NF):

Primary key: **(StudentID, ProjectID)**

Partial dependencies exist because student details depend only on StudentID, and project details depend only on ProjectID.

2NF decomposition:

Student(StudentID INT, StudentName VARCHAR(100), StudentMajor VARCHAR(100))

Project(ProjectID INT, ProjectTitle VARCHAR(150), ProjectType VARCHAR(50), SupervisorID INT, SupervisorName VARCHAR(100), SupervisorDept VARCHAR(100), StartDate DATE, EndDate DATE)

StudentProject(StudentID INT, ProjectID INT, Role VARCHAR(50), HoursWorked DECIMAL(6,1))

5. Third Normal Form (3NF):

Transitive dependency: ProjectID -> SupervisorID -> SupervisorName, SupervisorDept. Move supervisor details to a separate table.

Final 3NF schemas:

Student(StudentID INT, StudentName VARCHAR(100), StudentMajor VARCHAR(100))

Supervisor(SupervisorID INT, SupervisorName VARCHAR(100), SupervisorDept VARCHAR(100))

Project(ProjectID INT, ProjectTitle VARCHAR(150), ProjectType VARCHAR(50), *SupervisorID INT*, StartDate DATE, EndDate DATE)

StudentProject(StudentID INT, ProjectID INT, Role VARCHAR(50), HoursWorked DECIMAL(6,1))

Task 4.2: Advanced Normalization

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary key: **(StudentID, CourseID, InstructorID, TimeSlot, Room)**. Because the relation has no SectionID, the safest representation of one enrollment is the student plus the full set of attributes that identify the course section.

2. Functional dependencies:

- StudentID -> StudentMajor
- CourseID -> CourseName
- InstructorID -> InstructorName
- Room -> Building (rooms are unique across campus; therefore TimeSlot, Room also determines Building)

3. BCNF check:

The original table is not in BCNF. StudentID, CourseID, InstructorID, and Room determine non-key attributes, but none of them is a superkey of the full relation.

4. BCNF decomposition:

Student(StudentID INT, StudentMajor VARCHAR(100))

Course(CourseID INT, CourseName VARCHAR(150))

Instructor(InstructorID INT, InstructorName VARCHAR(100))

Room(Room VARCHAR(30), Building VARCHAR(100))

EnrollmentSchedule(StudentID INT, CourseID INT, InstructorID INT, TimeSlot VARCHAR(50), Room VARCHAR(30))

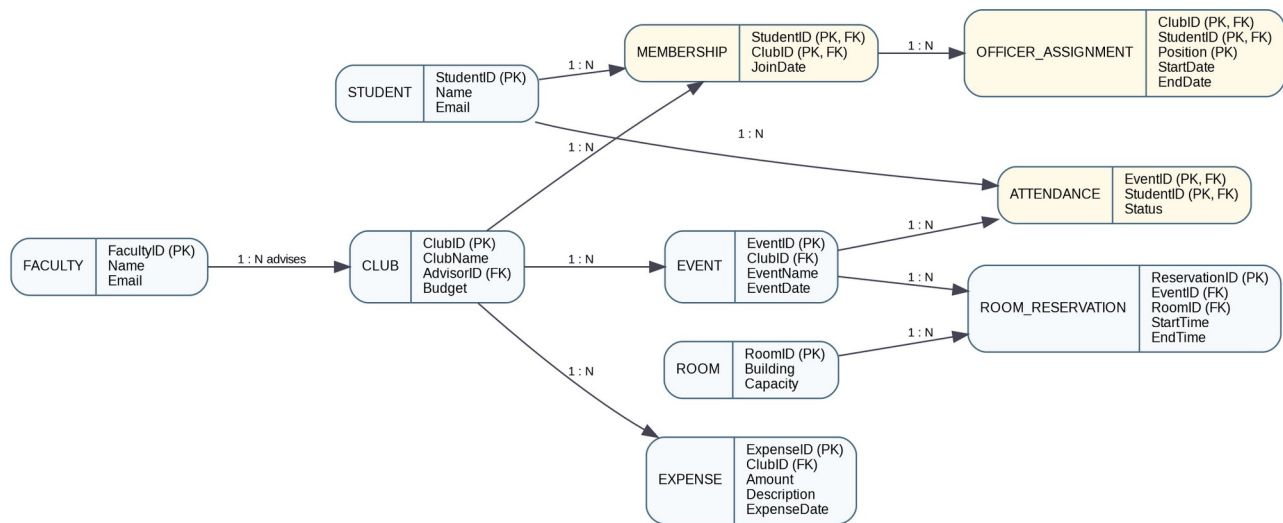
5. Information loss: **No information needs to be lost** if the decomposition is implemented with the listed primary/foreign keys. The original rows can be reconstructed by joining EnrollmentSchedule to Student, Course, Instructor, and Room. The residual EnrollmentSchedule keeps all section-identifying values.

Part 5: Design Challenge

Task 5.1: Real-World Application - Student Clubs

Design summary: students can join many clubs; clubs have many students; every club has one faculty advisor; clubs create events; attendance is tracked; officers are assignments of club members; rooms are reserved for events; club expenses are recorded against the club budget.

ER Diagram - Student Clubs and Organizations



Normalized Relational Schema

Student(StudentID INT, Name VARCHAR(100), Email VARCHAR(100))

Faculty(FacultyID INT, Name VARCHAR(100), Email VARCHAR(100))

Club(ClubID INT, ClubName VARCHAR(120), AdvisorID INT, Budget DECIMAL(12,2))

Membership(StudentID INT, ClubID INT, JoinDate DATE)

Event(EventID INT, ClubID INT, EventName VARCHAR(150), EventDate TIMESTAMP)

Attendance(EventID INT, StudentID INT, Status VARCHAR(20))

OfficerAssignment(ClubID INT, StudentID INT, Position VARCHAR(50), StartDate DATE, EndDate DATE)

Room(RoomID VARCHAR(20), Building VARCHAR(100), Capacity INT)

RoomReservation(ReservationID INT, EventID INT, RoomID VARCHAR(20), StartTime TIMESTAMP, EndTime TIMESTAMP)

Expense(ExpenseID INT, ClubID INT, Amount DECIMAL(10,2), Description VARCHAR(200), ExpenseDate DATE)

Table	Foreign key	References
Club	AdvisorID	Faculty(FacultyID)
Membership	StudentID	Student(StudentID)
Membership	ClubID	Club(ClubID)

Event	ClubID	Club(ClubID)
Attendance	EventID	Event(EventID)
Attendance	StudentID	Student(StudentID)
OfficerAssignment	(ClubID, StudentID)	Membership(ClubID, StudentID)
RoomReservation	EventID	Event(EventID)
RoomReservation	RoomID	Room(RoomID)
Expense	ClubID	Club(ClubID)

Design Decision

Room reservation was modeled as a separate table instead of storing RoomID directly in Event. This allows one event to have more than one reservation or time block and keeps room booking details separate from general event information.

Example Queries (English, not SQL)

1. Find all students who are officers in the Computer Science Club and show their officer positions.
2. List all club events scheduled for next week with their room reservations and room capacities.
3. Show the total expenses for each club and compare the amount with the club budget.

Submission Checklist

- All tasks from the provided assignment are answered.
- ER diagrams are included for Hospital, E-commerce, and Student Clubs.
- Normalization is shown step by step through 1NF, 2NF, 3NF, and BCNF where required.
- Relational schemas include primary keys, foreign keys, and data types.
- Primary-key attributes are bold and underlined in the schema definitions.